

Architecture - ScanMate

208127480	איתי צחור
B31	קוד פרוייקט
אסי ברק	מנחה

Abstract

ScanMate is a mobile application that bridges physical and digital chess through real-time computer vision. The system scans a physical chessboard via the device's camera, automatically detecting piece positions and tracking moves throughout the game. The architecture integrates a computer vision ML pipeline (running as a dedicated inference server), a Node.js backend with WebSocket support for real-time communication, and a React Native mobile client. A chess engine (Stockfish) is embedded to provide position evaluation, move suggestions, and post-game analysis. Users can save, replay, and export games to Lichess. The solution is designed for casual players, coaches, and chess clubs — improving accessibility, learning, and game record-keeping.

1. Introduction

ScanMate addresses this gap by providing a seamless way to capture and analyze physical chess games in real time. By leveraging machine learning and a mobile device camera, ScanMate identifies the state of the chessboard, records each move automatically, and integrates with advanced chess engines to deliver insights and evaluations.

The system is designed around three core principles:

- a responsive and intuitive mobile interface,

- a real-time computer vision pipeline for accurate board state detection,

- and efficient client-server communication for low-latency feedback.

The application merges the analytical power of digital tools with the experience of over-the-board play, making chess more engaging, educational, and accessible for all players.

2. Problem Statement

While modern online chess platforms provide automatic game recording, analysis tools, and integration with global chess communities, these features are largely inaccessible to players engaging in over-the-board chess. Physical chess games often remain undocumented because manually recording moves is time-consuming, error-prone, and disrupts the natural flow of play.

This lack of documentation leads to several challenges:

- Players lose the ability to review and learn from their games.
- Coaches and trainers cannot efficiently analyze their students' games.
- Clubs and tournament organizers face difficulties in recording games for archival or broadcast purposes.

Existing solutions for digitizing physical chess games are often limited, requiring specialized equipment, complex setup, or manual intervention. There is a clear need for a simple, accessible, and accurate tool that bridges the gap between physical and digital chess without disrupting gameplay. ScanMate addresses this by providing a modular, extensible architecture - separating computer vision, game logic, and user interface concerns - allowing the system to scale across different devices, game formats, and user needs.

3. Objectives

The main objectives of the ScanMate project are:

1. Real-time Chessboard Recognition - Develop a mobile application capable of detecting and interpreting the state of a physical chessboard using the device's camera.
2. Automatic Game Recording - Track and store all moves played during a game without manual input.
3. Integrated Chess Engine Analysis - Incorporate a powerful chess engine (e.g., Stockfish) to evaluate positions, suggest optimal moves, and provide post-game analysis.
4. Game Storage and Management - Enable users to maintain a personal library of past games, accessible anytime for review or sharing.
5. Platform Integration - Support exporting games in standard PGN format to Lichess.

6. Social Features - Allow users to invite friends and continue games remotely using scanned board positions.

4. User Stories

1. As a user I want to scan a current chessboard position, so I can instantly capture the game state without manual setup.
2. As a user I want to automatically record moves during a physical game, so I can review the complete game later.
3. As a user I want to sign up and log in, so I can access my personal game history and settings.
4. As a user, I want to get an analysis of the best move in the current chessboard position, so I can learn from my mistakes.
5. As a user I want to export games to Lichess, so I can access more users and use a better analysis.
6. As a user I want to save games to my personal library, so I can revisit and study them at any time.
7. As a user I want to replay games with a visual, move-by-move timeline, so I can understand mistakes and improvements.
8. As a user I want to save my games, so I can access them from different devices.
9. As a user I want to continue a scanned position online with a friend, so we can finish the game remotely.
10. As a user, I want to play with my friends online, so I can enjoy both physical and digital chess experiences.
11. As a user, I want to explore alternative moves from any position in a saved game, so I can understand what I could have done differently.

User Story 3: Authentication

```
// Verifies a signed-in user can retrieve their profile
async function testGetCurrentUser() {
  const res = await fetch(`${BASE_URL}/auth/me`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  const data = await res.json();
  console.log('Get user status:', res.status); // expect 200
  console.log('User:', data.user?.name);
}
```

User Story 6: Save Game

```
async function testSaveGame() {
  const res = await fetch(`${BASE_URL}/games`, {
    method: 'POST',
    headers: { Authorization: `Bearer ${token}`, 'Content-Type':
'application/json' },
    body: JSON.stringify({
      title: 'Test Game',
      moves: ['e4', 'e5', 'Nf3', 'Nc6'],
      startingFen: 'rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w
KQkq - 0 1',
      finalFen:
'r1bqkbnr/pppp1ppp/2n5/4p3/2B1P3/5N2/PPPP1PPP/RNBQK2R b KQkq - 3 3',
      result: '1-0',
      source: 'scan',
      opponentName: 'Opponent',
    }),
  });
  const data = await res.json();
  console.log('Save game status:', res.status); // expect 201
  gameId = data.game?._id;
  console.log('Saved game ID:', gameId);
}
```

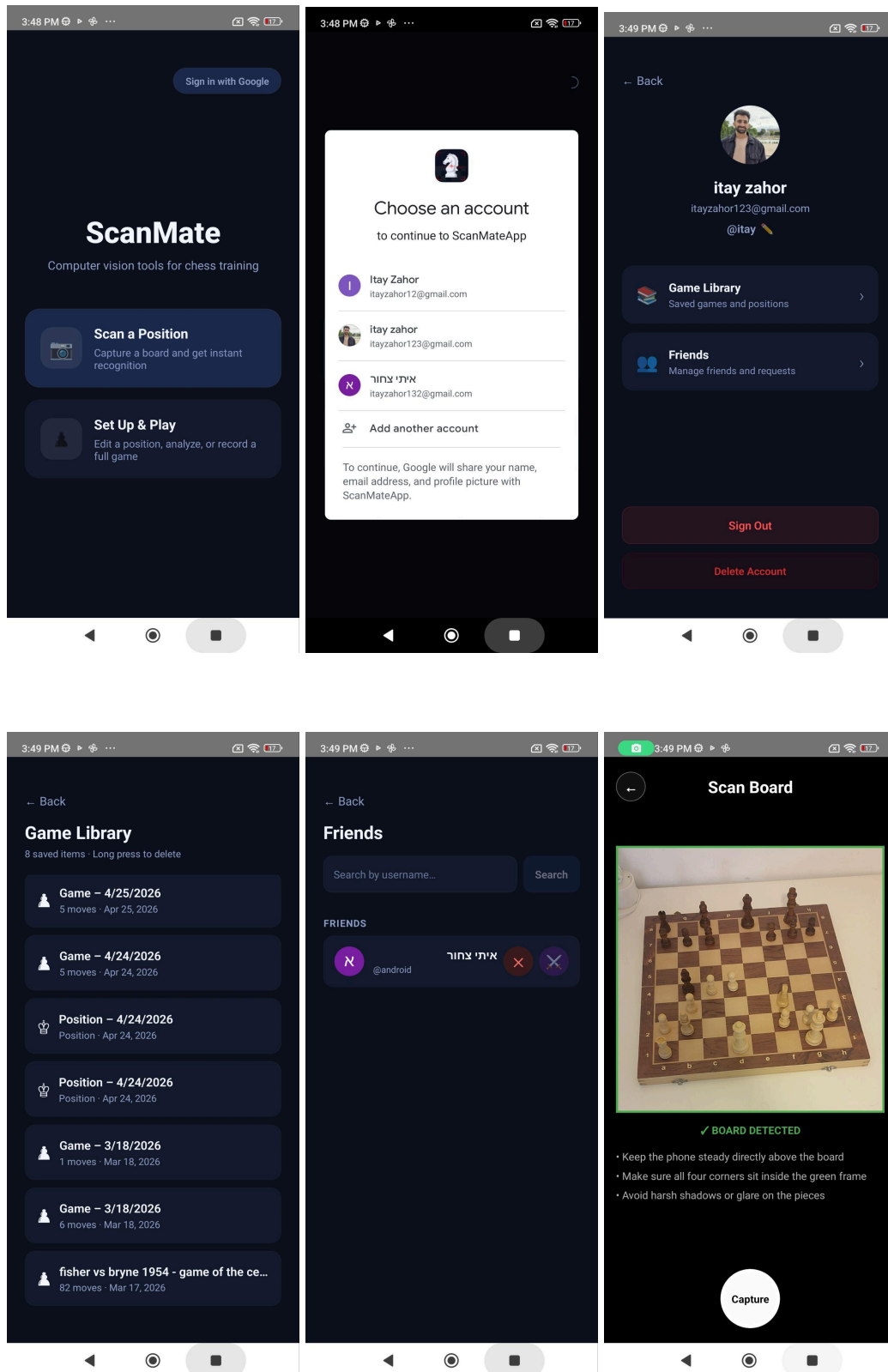
User Story 7: Get Game Library

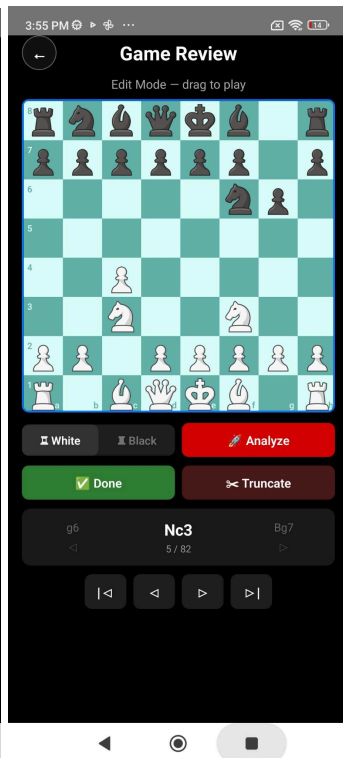
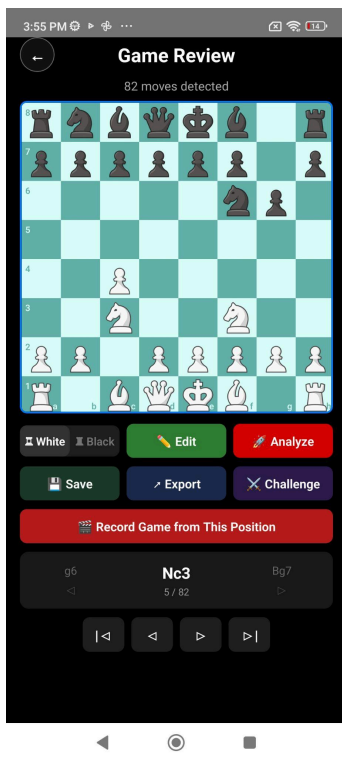
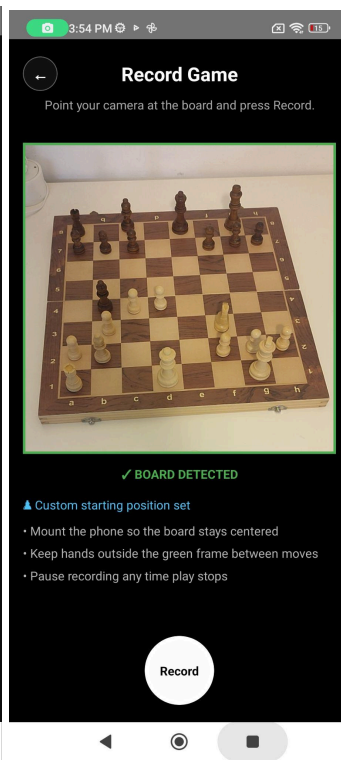
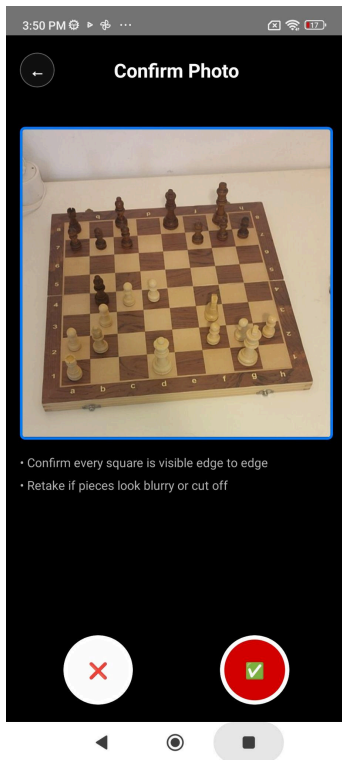
```
// Retrieve game library for replay – GET /api/games
async function testGetGameLibrary() {
  const res = await fetch(`${BASE_URL}/games?skip=0&limit=20`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  const data = await res.json();
  console.log('Get library status:', res.status); // expect 200
  console.log('Total games:', data.total);
}
```

User Story 9: Check for live games

```
// User Story 9: Check for active remote game session – GET
/api/live-games/active
async function testGetActiveLiveGame() {
  const res = await fetch(`${BASE_URL}/live-games/active`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  const data = await res.json();
  console.log('Active game status:', res.status); // expect 200
  console.log('Active game:', data.game ? data.game._id : 'none');
}
```

5. Application Screenshots





6. Typical User Flows

1. Scanning a Chess Position

- Open the ScanMate app.
- Tap "Scan a Position" on the home screen.
- Point the device camera above the chessboard and press Scan to capture the board state.
- The app processes the image and displays the detected FEN position.
- Optionally request engine evaluation of the board.

2. Recording a Full Game

- Open the ScanMate app.
- Tap "Set Up & Play".
- Press "Record Game from This Position"
- Scan the full game
- Review the game
- optionally save it to the personal library.

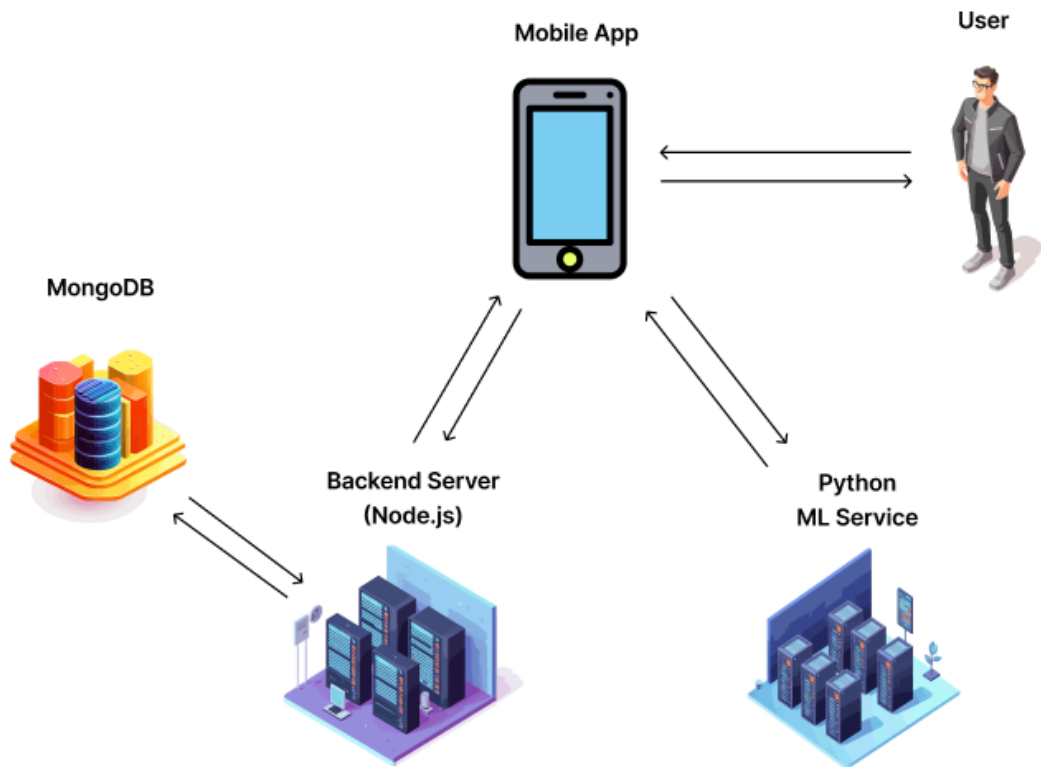
3. Viewing Past Games

- Open the ScanMate app.
- Tap the profile avatar (top-right) to navigate to the Profile screen, then select "Game Library".
- Browse the list of saved games
- Tap a game to open the move-by-move board replay (GameReview).

4. Signing In

- Open the ScanMate app.
- Tap "Sign in with Google".
- Complete the Google authentication flow.
- Upon success, the app loads the user's profile, game history, and personalized features.

7. Project Entity Diagram



8. Implementation

Mobile Frontend: React Native (TypeScript) — cross-platform iOS/Android app.

Backend Server: Node.js with Express — REST API and real-time communication via Socket.io.

ML Service: Python with FastAPI — computer vision pipeline for board and piece detection, Stockfish engine integration.

Database: MongoDB (via Mongoose) — stores users, games, friendships, and live game sessions.

Chess Engine: Stockfish — position evaluation and best-move suggestions.

Chess Logic: chess.js — move validation, FEN/PGN parsing, game tree management (used on both client and server).

Authentication: Google OAuth 2.0 + JWT — secure sign-in without email/password management.

Computer Vision Models: YOLOv11 — two custom-trained models for board corner detection and piece classification.